コンパイラ (11)

2026/11/10

データフロー解析

基本ブロック B に対するデータフロー (定義到達解析)

定義: $x=e_1 \oplus e_2$, $x=e$
左辺で出現した変数がどこまで更新されずに右辺で参照されるか？

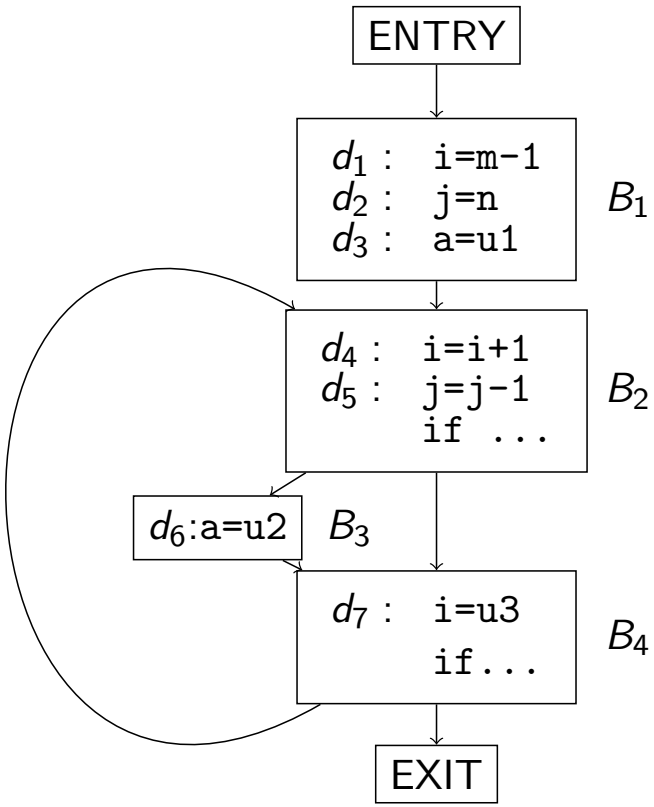
$$out[B] = gen[B] \cup (in[B] - kill[B])$$

プログラム上での位置 (行番号)：

$in[B]$	B に入力されるデータ
$out[B]$	B から出力されるデータ
$gen[B]$	B で定義されるデータ
$kill[B]$	B で上書きされるデータ

入力から出力への制御がきまっていればよいので、基本ブロック単位で考えればよい。
ループ：データフロー方程式を解いて全体的に必要なデータを計算する。

データフロー方程式



$$gen_{B_1} = \{d_1, d_2, d_3\}$$
$$kill_{B_1} = \{d_4, d_5, d_6, d_7\}$$

$$gen_{B_2} = \{d_4, d_5\}$$
$$kill_{B_2} = \{d_1, d_2, d_7\}$$

$$gen_{B_3} = \{d_6\}$$
$$kill_{B_3} = \{d_3\}$$

$$gen_{B_4} = \{d_7\}$$
$$kill_{B_4} = \{d_1, d_4\}$$

$$IN[B_2] = OUT[B_1] \cup OUT[B_4]$$
$$OUT[B_2] = gen_{B_2} \cup (IN[B_2] - kill_{B_2})$$

(Aho ら：Compilers より引用)

データフロー方程式

$$
\begin{aligned}
OUT[B_1] &= gen_{B_1} \\
IN[B_2] &= OUT[B_1] \cup OUT[B_4] \\
OUT[B_2] &= gen_{B_2} \cup (IN[B_2] - kill_{B_2}) \\
IN[B_3] &= OUT[B_2] \\
OUT[B_3] &= gen_{B_3} \cup (IN[B_3] - kill_{B_3}) \\
IN[B_4] &= OUT[B_3] \cup OUT[B_2] \\
OUT[B_4] &= gen_{B_4} \cup (IN[B_4] - kill_{B_4})
\end{aligned}
$$

データフロー方程式の計算

到達可能性解析：

XXXX XXX:$d_1, d_2, \cdots, d_7$ が集合に入っているとき 1、入ってないとき 0

Block B	OUT$[B]^0$	IN$[B]^1$	OUT$[B]^1$	IN$[B]^2$	OUT$[B]^2$
B_1	111 0000	000 0000	111 0000	000 0000	111 0000
B_2	000 1100	111 0001	001 1100	111 0111	001 1110
B_3	000 0010	000 1100	000 1110	001 1100	000 1110
B_4	000 0001	000 1110	000 0111	001 1110	001 0111

変化しなくなるまでくりかえす。

$OUT[B_4]$=001 0111:

本ブロックでは d_3, d_5, d_6, d_7 が EXIT が到達する可能性がある.

データフロー

生存解析

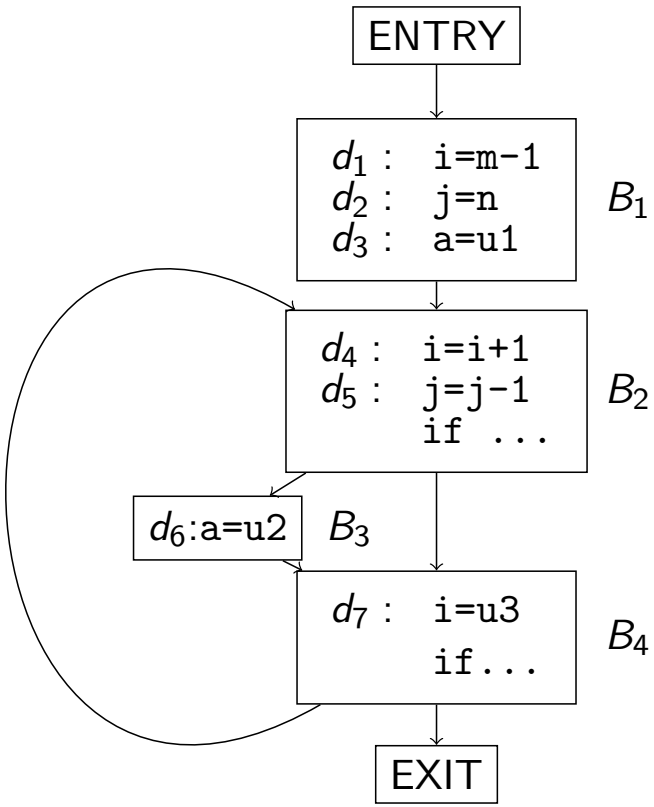
$Use[B]$: B で再定義される前に資料される変数の集合
$Def[B]$: B で定義される変数の集合

$$
\begin{aligned}
In[B] &= Use[B] \cup (Out[B] - Def[B]) \\
Out[B] &= \cup_{S \in Succ[B]} In[S]
\end{aligned}
$$

$Succ[B]$: フローグラフにおいて B から直接フローがあるブロック

フローグラフ下流の方から計算する

データフロー解析



$$Def_{B_1} = \{i, j, a\}$$
$$Use_{B_1} = \{m, n, u_1\}$$

$$Def_{B_2} = \{i, j\}$$
$$Use_{B_2} = \{i, j\}$$

$$Def_{B_3} = \{a\}$$
$$Use_{B_3} = \{u2\}$$

$$Def_{B_4} = \{i\}$$
$$Use_{B_4} = \{u3\}$$

データフロー解析（生存解析）

$(a, i, j, m, n, u1, u2, u3)$ とすると

	OUT	IN	OUT	IN	OUT	IN
B_1	(00000000)	(00011100)	(01100000)	(00011100)	(01100010)	(00011100)
B_2	(00000000)	(01100000)	(01100010)	(01100010)	(01100001)	(01100010)
B_3	(00000000)	(00000010)	(00000001)	(00000010)	(00100001)	(10100001)
B_4	(00000000)	(00000001)	(01100000)	(00100001)	(01100010)	(00100001)

フローグラフの基本ブロックにおいてどの変数が影響をうけるかを解析

	IN	OUT
B_1	$\{m, n, u1\}$	$\{i, j, u2\}$
B_2	$\{i, j, u2\}$	$\{i, j, u3\}$
B_3	$\{a, j, u3\}$	$\{j, u3\}$
B_4	$\{j, u3\}$	$\{i, j, u2\}$

データフロー解析（共通部分式）

部分式に含まれるすべてのフローをチェックする．
例えば，v=a+b における a+b の計算が重複されているかどうかは，すべてのフローが同じであることをチェックする．

$$IN[B] = \bigcap_{\mathrm{Pred}[B]} Out[P] \quad (1)$$

$$OUT[B] = Gen[B] \cup (In[B] \backslash Kill[B]) \quad (2)$$

静的単一代入を用いた最適化

データフロー解析のもうひとつの手法: 静的単一代入 (Static Single Assignment:SSA)

各変数の定義がプログラム中で唯一

変数の値は変わらないとする。二度目の代入が発生すると変数名を変更

　値が同じの共通部分式が判別できる=式として一致

変換例：例 6.20、6.21

定義ごとに変数に添字をつける。合流する場合は、ϕ 関数で表現

いかにして、SSA 形式を得るか？

(1) ϕ 関数の挿入

(2) 変数の添字の決定

SSA 形式において最適化

(3) SSA から逆変換する。(ϕ 関数の消去)

近年のコンパイラでよく用いられる手法

例

```
scanf("%d",&x);
a=1;
b=2;
c=a+b;
if(x>0) a=3;
d=b;
c=a+b;
```

⇒

```
scanf("%d",&x0);
a0=1;
b0=2;
c0=a0+b0;
if(x>0) a1=3;
a2=φ(a0,a1);
d0=b0;
c1=a2+b0;
```

```
t=1
n=10
l1:  t=t*n
n=n-1
b=n>1
if b goto l1
f=t
```

⇒

```
    t1=1
    n1=10
l1: t2=φ(t1,t3)
    n2=φ(n1,n3)
    t3=t2*n2
    n3=n2-1
    b1=n3>1
    if b1 goto l1
    f1=t3
```

φ 関数の挿入

フローグラフ (V, E, v_{init})：$v \in V$ は，v_{init} から到達可能．

定義 7.1(定義合流条件)　変数 x に対する φ 関数が頂点 v に挿入 $\Leftrightarrow$ 以下に示す条件 (1)〜(4) を満たすコントロールグラフ上の路 $p : v_0 \to v_1 \to \cdots \to v_k$, $q : u_0 \to u_1 \to \cdots \to u_\ell$ が存在．

(1) 路の長さが 1 以上である．$(k > 0, \ell > 0)$;

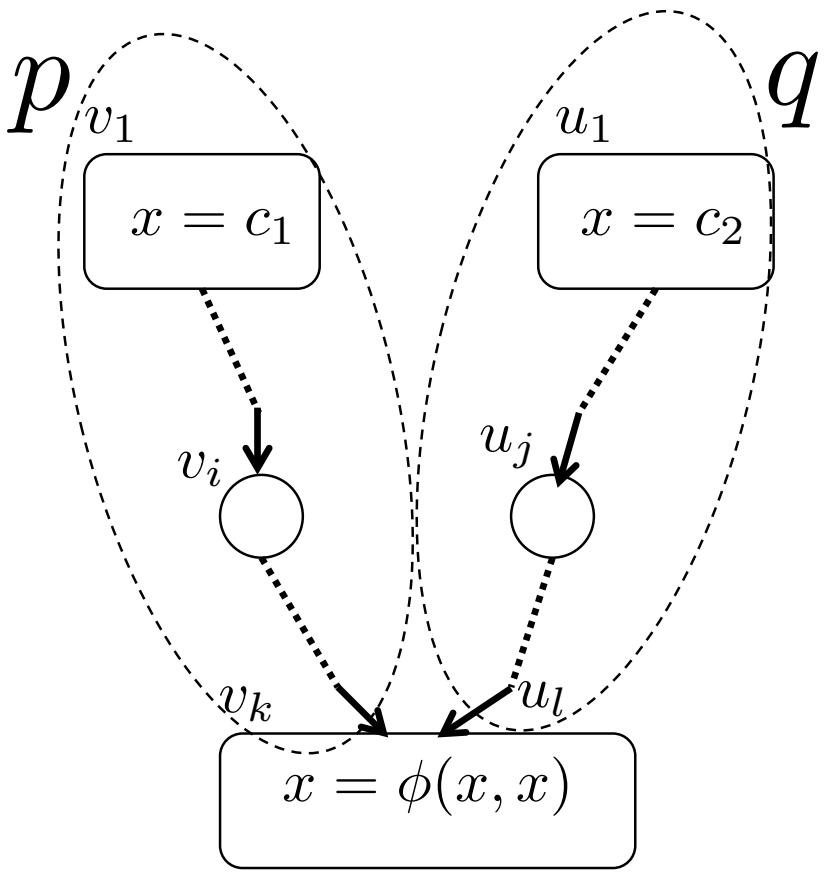
(2) v_1 と u_1 の両方に x の定義が存在する．;

(3) 路の終点 v が一致している．$(v_k = v_\ell = v)$;

(4) 路の途中で交わることがない．
$\left(\{v_1, \cdots, v_{k-1}\} \cap \{u_1, \cdots, v_{\ell-1}\} = \varnothing\right)$

パス p 上の x の定義$=x_1$,q 上の x の定義$=x_2$ 定義合流条件の v に、$x_3 = \phi(x_1, x_2)$

定義合流条件



支配関係

支配関係：開始頂点からのパスに必ず含まれるとき、「支配」される。

定義 7.2(支配関係)　フローグラフを (V, E, v_{init}) とする．頂点 v, u に対して，v_{init} から u へのすべての路において v が出現するとき，v は u の支配頂点 (dominator) であると呼び，$u \underline{\ll} v$ と書く．$u \neq v$ かつ $u \underline{\ll} v$ であるとき，$u \ll v$ と書く．u の支配頂点の集合 $\mathrm{dom}(u) = \{v \mid u \underline{\ll} v\}$ で定義する．

$u \underline{\gg} v$:u が v を支配: 支配関係の表現=支配木

支配関係

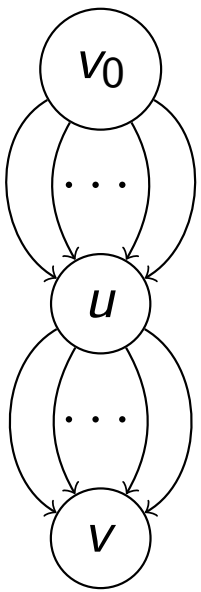
支配関係：開始頂点からのパスに必ず含まれるとき、「支配」される。
どの頂点 v についても $v_{init} \in \mathrm{dom}(v)$ かつ $v \in \mathrm{dom}(v)$ である．$\underline{\ll}$ は半順序である．

- (反射律) $v \underline{\ll} v$
- (反対称律) $v \underline{\ll} u$ かつ $u \underline{\ll} v$ ならば $u = v$
- (推移律) $v \underline{\ll} u$ かつ $u \underline{\ll} w$ ならば $v \underline{\ll} w$

$u \neq v_{init}$ のとき，$\mathrm{dom}(u) \backslash \{u\}$ には $\underline{\ll}$ の意味で最小の頂点が一つ存在する．

支配関係

支配関係：開始頂点からのパスに必ず含まれるとき、「支配」される。



$$u \underline{\gg} v$$

$$u \underline{\gg} u$$

$u \underline{\gg} v, v \underline{\gg} w$ ならば $u \underline{\gg} w$

$u \neq v$ のとき $u \gg v$

v 以外で最も下にある支配頂点 u: $idom(v)$

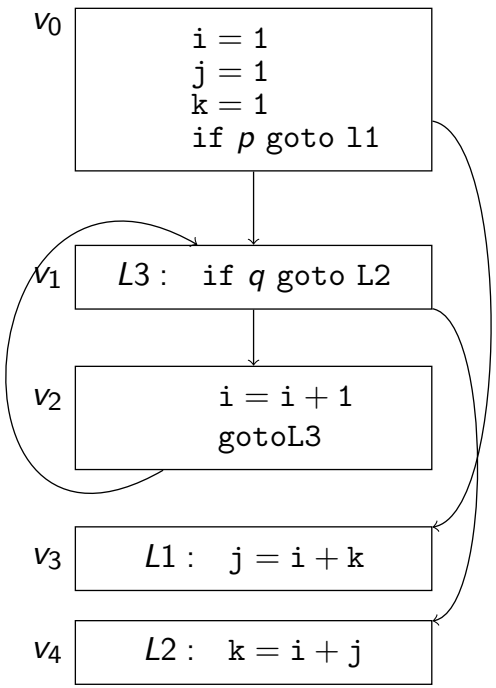
$(idom(v), v)$ を辺とするグラフ＝支配木

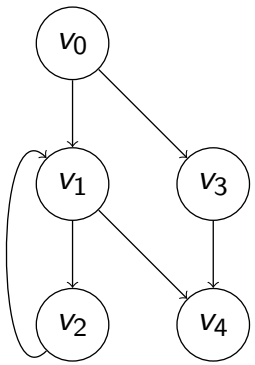
厳密な SSA 形式

SSA 形式が満たす性質

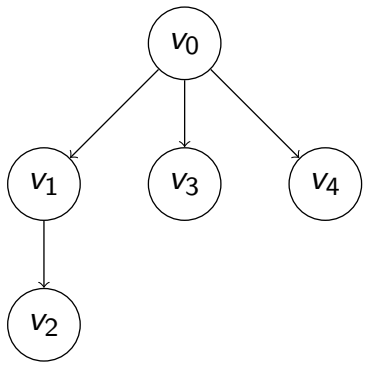
1. 定義式の左辺の変数がプログラム中で唯一
   構文的に左辺の変数は唯一
2. 右辺に現れる変数は最も近い上流の定義式で更新される
   更新をとびこえて参照されることはない．(支配関係の保存)

フローグラフ → 支配木





フローグラフ



支配木

静的単一代入を用いた最適化

データフロー解析のもうひとつの手法:
静的単一代入 (Static Single Assignment:SSA)
各変数の定義がプログラム中で唯一
二度目の代入が発生すると変数名を変更
　値が同じの共通部分式が判別できる=式として一致
定義ごとに変数に添字をつける。合流する場合は、φ 関数で表現
いかにして、SSA 形式を得るか？

(1) φ 関数の挿入
(2) 変数の添字の決定

　　SSA 形式において最適化

(3) SSA から逆変換する。(φ 関数の消去)

近年のコンパイラでよく用いられる手法

例

```
scanf("%d",&x);
a=1;
b=2;
c=a+b;
if(x>0) a=3;
d=b;
c=a+b;
```

⇒

```
scanf("%d",&x0);
a0=1;
b0=2;
c0=a0+b0;
if(x>0) a1=3;
a2=φ(a0,a1);
d0=b0;
c1=a2+b0;
```

```
t=1
n=10
l1:  t=t*n
n=n-1
b=n>1
if b goto l1
f=t
```

⇒

```
    t1=1
    n1=10
l1: t2=φ(t1,t3)
    n2=φ(n1,n3)
    t3=t2*n2
    n3=n2-1
    b1=n3>1
    if b1 goto l1
    f1=t3
```

φ 関数の挿入

$\mathtt{x}=\varphi(\mathtt{x},\cdots,\mathtt{x})$ の挿入：

- 変数 $\mathtt{x}$ に対して定義 *7.1* の条件を満たす v_0，u_0，v がフローグラフに存在し，頂点 v の基本ブロックに φ 関数が含まれていないならば，v に $\mathtt{x}=\varphi(\mathtt{x},\ldots,\mathtt{x})$ を挿入する．
- 定義合流条件を満たす v_0，u_0，v がなくなるまで，上記を繰り返す．